DEI
DEPARTAMENTO
DE ENGENHARIA INFORMÁTICA
TÉCNICO LISBOA

Apresentação

Introdução aos Algoritmos e Estruturas de Dados

IAED

Corpo Docente

- **Docentes**
  - Pedro Adão (teóricas)
  - Vasco Manquinho (labs)
  - Guilherme Ramos (labs)
  - Emanuel Gonçalves (labs)

- **Horários de Dúvidas**
  - Afixados na página da cadeira

- **Mattermost**:
  - Link na FAQ da disciplina

Carga Horária

- **Aulas Teóricas:** 5 horas semanais
- **Aulas de Laboratório:** 2 aulas semanais de 1h30
  - Alunos devem frequentar os turnos em que estão inscritos

Aulas de Laboratório

- O trabalho nas aulas laboratoriais **será individual**!
- Os alunos **têm de comparecer** no turno laboratorial em que se inscreveram

- Aulas divididas em duas partes:
  - Resolução de exercícios
  - Apoio a cada aluno na resolução do guia de laboratório

- Guia de laboratório disponibilizado antes para **preparação da aula**
- Será possível testar as vossas soluções dos laboratórios no **GitLab**
  - Detalhes disponibilizados nos laboratórios e na página

Projecto

- O projecto será realizado **individualmente**
- No final do semestre haverá um **teste prático individual** sobre o projecto. A nota final da componente de projecto depende deste teste prático
- Utilização de sistema automático para entrega de projectos
  - Repositório individual **GitLab** com avaliação automática
- **Não serão aceites entregas por email!**

Projectos

- **Projecto (P):** ***2 entregas***; nota mínima de 7 valores; nota de cada aluno ponderada pela nota do teste prático
  - 1ª Entrega (P1): 18 de Março
  - Entrega final (P2): 06 de Abril
  - Teste Prático (TP): 08 de Abril

Avaliação

Exame (E)

- Nota mínima de 7 valores

Nota provisória do Projecto (P)

- **P = 0.4*P1 + 0.6*P2**

Nota do Final do Projecto (NP)

- **NP = min(P,TP) + sqrt(|P-TP|/2))**
- Nota mínima de 7 valores

Nota Final (NF = 0.5 * NP + 0.5 E)

- **Arredondamentos apenas na nota final**

TP = Teste Prático

Avaliação

- Anos Anteriores – Projectos
  - Todos os alunos a frequentar a disciplina para aprovação ou que pretendam fazer melhoria de classificação **serão avaliados em todas as componentes de avaliação**

- Alunos não inscritos
  - Os alunos não inscritos não serão avaliados

Avaliação

- Não serão tolerados quaisquer tipos de fraude em qualquer componente da avaliação
  - Exemplos:
    - Obtenção do enunciado antes de prova
    - Cópias totais ou parciais de funções/projecto
    - Fornecer código do projecto a outro grupo
    - ...
- Comunicação da tentativa de fraude aos orgãos do IST
- Penalizações serão as maiores possíveis de acordo com o regulamento do IST

Programa

- Introdução à linguagem de programação C – Parte I
- Introdução ao estudo da eficiência de algoritmos
- Algoritmos de ordenação: inserção directa, selecção directa, bubblesort, quicksort, heapsort e radixsort
- Introdução à linguagem de programação C – Parte II
- Estruturas de dados: pilhas, filas de espera, filas de prioridade, amontoados, árvores e tabelas de dispersão

Bibliografia Recomendada

- Programação em C
  - B. Kernighan e D. Ritchie*, The C Programming Language*, 1988, Prentice Hall

- Estruturas de Dados e Algoritmos de Ordenação
  - R. Sedgewick, *Algorithms in C: Parts 1-4*, 1998, Addison-Wesley Publishing Company

- Algoritmos elementares e Grafos
  - T. Cormen, C. Leiserson, R. Rivest, C. Stein, *Introduction to Algorithms*, 2009, MIT Press
  - Livro de referência em Análise e Síntese de Algoritmos

Material de Apoio

- **Teóricas:** slides das aulas teóricas, afixados depois da aula ter decorrido.
- **Laboratórios:** enunciados dos exercícios de laboratório para cada semana (publicados com antecedência para preparação prévia da aula)
- **Projecto:** enunciados dos projectos, ficheiros de exemplo, etc.
- **Avaliação:** enunciados e resoluções de testes de anos anteriores
- **FAQ**

Dúvidas?

DEI
DEPARTAMENTO
DE ENGENHARIA INFORMÁTICA
TÉCNICO LISBOA

Introdução à Linguagem C

K&R: Capitulo 1

IAED

Introdução

- Desenvolvida em 1972 por Dennis Ritchie, nos Bell Labs, para utilização no sistema operativo UNIX
- O standard ANSI C (ISO/IEC 9899:1990) foi adoptado pela ISO em 1990
- Linguagem de alto nível, mas que permite acesso de baixo nível a memória e dispositivos
- A maioria dos sistemas operativos actuais (Linux, Windows, MacOS, etc) continua a ser programado em C
- Influenciou o desenvolvimento de diversas linguagem como: Java, C++, C#, Perl, PHP, JavaScript, etc

Introdução

- Introdução rápida à linguagem C, utilizando exemplos:
  - Programa que escreve `hello, world`
  - Conversão de temperaturas
  - Cópia de ficheiros
  - Contagem de caracteres
  - Contagem de linhas
  - Contagem de palavras

- Conteúdos
  - Tabelas
  - Funções
  - Passagem por valor
  - Tabelas de caracteres
  - Variáveis externas

hello, world

```
#include <stdio.h>

main()
{
  printf("hello, world\n");
}
```

hello, world

```
#include <stdio.h>

main()
{
   printf("hello, world\n");
}
```

- Biblioteca de funções de entrada/saída

hello, world

```
#include <stdio.h>

main()
{
   printf("hello, world\n");
}
```

- Nome de uma função pode ser qualquer
- Todos os programas têm uma função **`main`**
  - A função `main` é a primeira a ser executada
- Funções podem ser definidas em múltiplos ficheiros (a ver mais à frente no período)

hello, world

```
#include <stdio.h>

main()
{
   printf("hello, world\n");
}
```

- Parâmetros formais da função
- Comunicação do exterior com a função
- Neste caso não existem parâmetros

hello, world

```
#include <stdio.h>

main()
{
  printf("hello, world\n");
}
```

- Instruções associadas a uma função são agrupadas com chavetas
  - vs. Python, onde a indentação influencia a semântica das funções

hello, world

```
#include <stdio.h>

main()
{
   printf("hello, world\n");
}
```

- Nesta função temos apenas uma instrução
  - Chamada à função `printf`
- Parâmetros actuais da função `printf` colocados entre parêntesis
- Instruções separadas por `;`

hello, world

```
#include <stdio.h>

main()
{
   printf("hello, world\n");
}
```

- Cadeia de caracteres entre aspas vs. Python, onde também podem ser usadas aspas simples para delimitar strings
- Outros caracteres
  - `\n` Sequência de caracteres que representa *newline*
  - `\t` *tab*
  - `\b` *backspace*
  - `\"` aspas
  - `\\` barra

hello, world

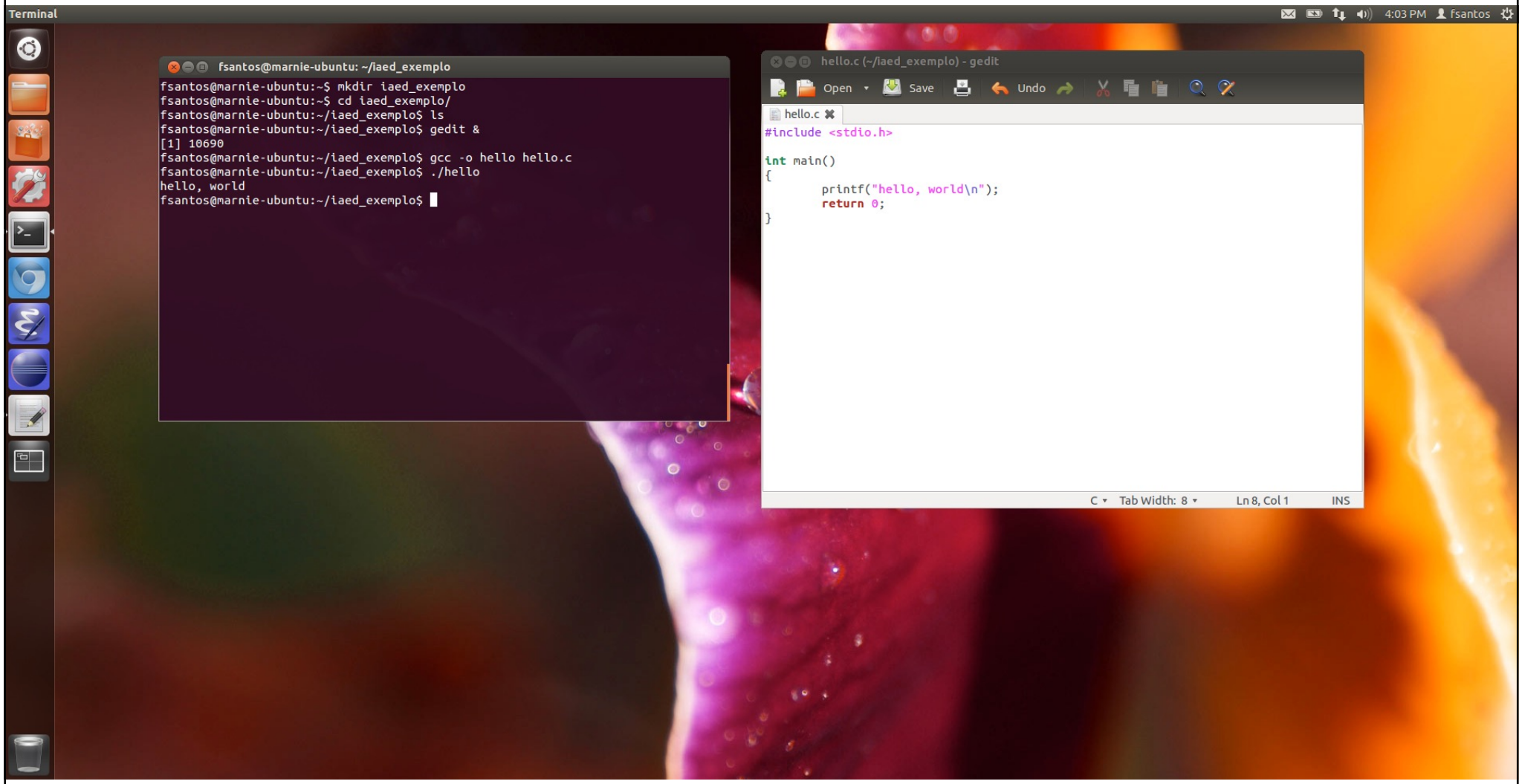
```
#include <stdio.h>

int main()
{
   printf("hello, world\n");
   return 0;
}
```

- Vamos compilar este programa. Como fazer? Onde devo escrever o meu código? Em que sistema operativo?

(já voltaremos aos tipos)

Ambiente UNIX (e.g. LINUX)



Laboratórios: Linux (Dist. OpenSuse)

Sugestão de Ambiente de trabalho

- Configurar um ambiente de trabalho para a cadeira de IAED nos vossos computadores pessoais. Sugestão:
  - Instalar & configurar uma distribuição Linux (e.g., Ubuntu de modo a ficarem com as mesmas configurações dos laboratórios & compilador *gcc* usado na avaliação dos projectos)
  - **Instalar Visual Studio Code**
  - **Instalar cliente GIT (ver FAQ)**
  - Se por qualquer razão preferir usar outro sistema operativo (e.g., MacOS [OK!], Windows [A evitar!]) procurem encontrar uma forma de usar o gcc
    - No Mac instale o "Xcode"
    - no Windows instale WSL (Ubuntu)

Editores de texto

- Editores de texto simples (Unix)
  - Geany (all), gedit (GNOME/Linux), Sublime (all), kate (KDE/Linux), jedit (all), emacs (all), vi (all), textmate (MacOS), Aquamacs (MacOS), **Visual Studio Code (all)**, etc.

- IDEs (Integrated development environment, Linux, Mac & Windows)
  - **Visual Studio Code**
  - Eclipse IDE for C/C++ developers
  - Code::Blocks, etc.

Linux command line (exemplos)

- `$ mkdir iaed_test` (cria uma directoria)
- `$ cd iaed_test` (muda de directoria)
- `$ cd ..` (desce uma directoria)
- `$ ls` (lista os ficheiros e directorias nessa posição)
- `$ code &`
  (o `&` chama o editor e coloca-o a executar em background sem bloquear o terminal)

Ver mais exemplos aqui:
http://linuxcommand.org/

Linux command line (exemplos)

Compilar um programa com o **gcc**

- `$ gcc -o hello hello.c` (cria um executável `hello` compilando o código contido no ficheiro `hello.c`)
- `$ ./hello` (correr o executável!)

resultado

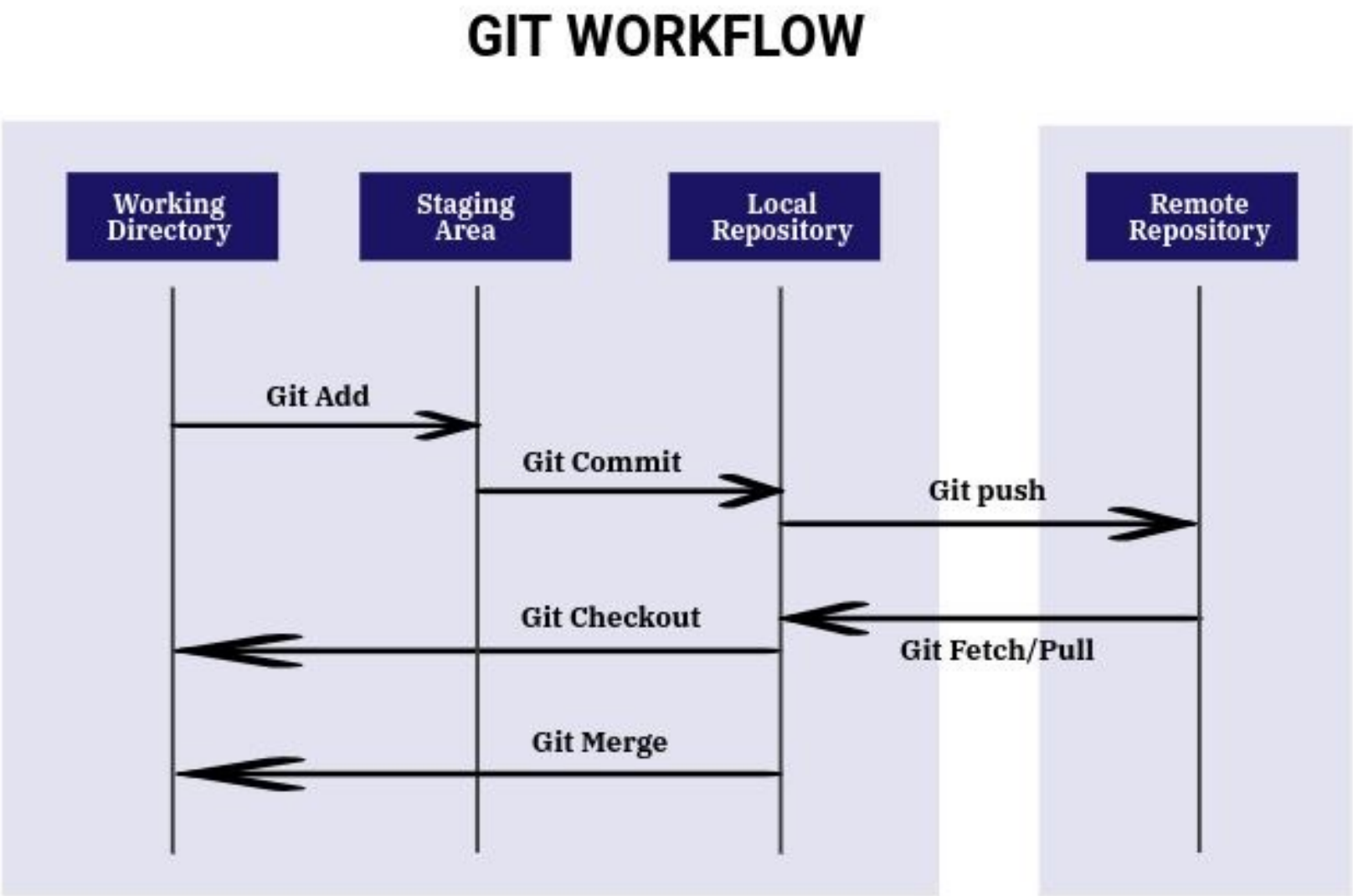
```
$ ./hello
hello, world
$
```

Utilização de Git (Labs. e projecto)

- Cada aluno terá um repositório **Git** individual para os laboratórios e o projecto
- Git é um sistema distribuido de controlo de versões
- Submissões do projecto serão feitas utilizando Git
  - A última versão que estiver no repositório será a versão avaliada
  - Não serão avaliadas submissões por outras vias

- Iremos utilizar o GitLab da RNL
  - http://gitlab.rnl.tecnico.ulisboa.pt

Utilização de Git (Labs e projecto)



Novidades em C

```python
def factorial(n):
  if n == 0:
    return 1
  else:
    return n*factorial(n-1)
```

Python

```c
int factorial (int n)
{
  if (n == 0) {
    return 1;
  }
  else {
    return n*factorial(n-1);
  }
}
```

C

- Tipificação estática (C não permite tipificação dinâmica)
  - Tipo de cada variável definido na compilação

Novidades em C

```python
def factorial(n):
  if n == 0:
    return 1
  else:
    return n*factorial(n-1)
```

Python

```c
int factorial (int n)
{
  if (n == 0) {
    return 1;
  }
  else {
    return n*factorial(n-1);
  }
}
```

C

- Tipificação estática (C não permite tipificação dinâmica)
  - Tipo de cada variável definido na compilação
- Blocos delimitados por { } em vez de indentação
  - Indentação é apenas utilizada para tornar o código mais legível
  - Se o bloco só contém uma instrução as { } são opcionais

Novidades em C

Python

```python
def factorial(n):
  if n == 0:
    return 1
  else:
    return n*factorial(n-1)
```

C

```c
int factorial (int n)
{
if (n == 0) {
return 1;
}
else {
return n*factorial(n-1);
}
}
```

Assim também funciona!

- Tipificação estática (C não permite tipificação dinâmica)
  - Tipo de cada variável definido na compilação
- Blocos delimitados por { } em vez de indentação
  - Indentação é apenas utilizada para tornar o código mais legível
  - Se o bloco só contém uma instrução as { } são opcionais

Novidades em C

Python

```
def factorial(n):
  if n == 0:
    return 1
  else:
    return n*factorial(n-1)
```

C

```
int factorial (int n)
{ if (n == 0) {return 1;}
else {return n*factorial(n-
   1);}}
```

Assim também !!!!

- Tipificação estática (C não permite tipificação dinâmica)
  - Tipo de cada variável definido na compilação
- Blocos delimitados por { } em vez de indentação
  - Indentação é apenas utilizada para tornar o código mais legível
  - Se o bloco só contém uma instrução as { } são opcionais

Factorial

```
int factorial (int n) {
  if (n == 0)
    return 1;
  else
    return n*factorial(n-1);
}
```

- Tipo de dados do valor de retorno da função
  - Precede o nome da função
- Tipos básicos: `char`, `int`, `long int`, `double` e `float`;
- Tipos complexos: estruturas, uniões, tabelas, ponteiros para tipos mais simples e funções

Factorial

```
int factorial (int n) {
  if (n == 0)
    return 1;
  else
    return n*factorial(n-1);
}
```

- Tipo de dados do parâmetro `n`
- Todos os parâmetros são precedidos pelo seu tipo

Factorial

```
int factorial (int n) {
  if (n == 0)
    return 1;
  else
    return n*factorial(n-1);
}
```

- Sintaxe da estrutura de controlo: `if-then-else`

```
if (<condição>)
  <instrução1>
else
  <instrução2>
```

- Se a `condição` é verdadeira, executa `instrução1`
- Caso contrário, executa `instrução2`

Factorial

```
int factorial (int n) {
  if (n == 0)
    return 1;
  else
    return n*factorial(n-1);
}
```

- Sintaxe da instrução `return`

  `return <expressão>;`
- Termina a execução da função
- Resultado da `expressão` é o valor de retorno da função